

Universidad Distrital Francisco José de Caldas

Systems Engineering Program
Faculty of Engineering

Technical Report

Ecosystem Survival

Cristian Esteban Castañeda Vargas
Julian Carvajal Garnica
Juan Pablo Angulo Guerrero

Group: 8

Supervisor: Carlos A. Sierra Virgüez

Bogotá, Colombia
June 4, 2026

Declaration

We, Cristian Esteban Castañeda Vargas, Julian Carvajal Garnica, and Juan Pablo Angulo Guerrero, students of the Systems Engineering Program, Faculty of Engineering at Universidad Distrital Francisco José de Caldas, declare that this report is entirely our own work.

All figures, tables, code fragments, and illustrations included in this document are original, except where properly cited and referenced. We understand that failure to acknowledge sources constitutes plagiarism, which is considered academic misconduct.

We give consent for this report to be shared with future students as a reference.

We also give consent for this work to be made available for educational and research purposes.

Cristian Esteban Castañeda Vargas
Julian Carvajal Garnica
Juan Pablo Angulo Guerrero

June 4, 2026

Contents

List of Figures	iv
List of Tables	v
List of Abbreviations	vi
Abstract	1
1 Introduction	2
1.1 Background	2
1.2 Problem Statement	3
1.3 Aims and Objectives	3
1.4 Solution Approach	3
1.5 Scope	4
1.6 Assumptions	4
1.7 Limitations	4
1.8 Summary of Contributions and Achievements	5
1.9 Organization of the Report	5
2 Literature Review	6
2.1 Software Architecture for Digital Game Mechanics	6
2.2 Algorithms in Game Decision-Making	6
2.3 Backtracking and Constrained Pathfinding	7
2.4 Data Structures for Game State Management	7
2.5 JSON as a Communication Contract	7
2.6 Critique of the Review	7
2.7 Summary	8
3 Methodology	9
3.1 General Design Approach	9
3.2 System Architecture	9
3.3 Game State Representation	10
3.4 Game Rules and Turn System	10
3.5 Greedy Algorithm Methodology	11
3.6 Backtracking Algorithm Methodology	11
3.7 C++ Engine Methodology	11
3.8 Linked List Methodology	12
3.9 AVL Tree Methodology	12
3.10 JSON Bridge Methodology	12
3.11 User Interface Methodology	13
3.12 Console Simulator Methodology	13
3.13 Validation Methodology	14

3.14 Summary	14
4 Implementation and Results	15
4.1 Implemented Components	15
4.2 Gameplay From the User Point of View	15
4.3 Greedy Algorithm Result	15
4.4 Backtracking Algorithm Result	16
4.5 Contribution of the C++ Engine	16
4.6 Use of JSON Files	16
4.7 Validation	17
5 Discussion	18
5.1 Interpretation of the Implementation	18
5.2 Greedy Algorithm Analysis	18
5.3 Backtracking Analysis	18
5.4 Linked List Analysis	19
5.5 AVL Tree Analysis	19
5.6 JSON Bridge Analysis	19
5.7 Technical Challenges	20
5.8 Scalability Considerations	20
5.9 Future Improvements	20
6 Conclusions	21
References	23

List of Figures

List of Tables

4.1 Validation summary based on the inspected repository.	17
---	----

List of Abbreviations

AVL	Adelson-Velsky and Landis tree; a self-balancing binary search tree used to store species data.
BFS	Breadth-First Search; a possible future alternative for finding shortest migration paths.
DFS	Depth-First Search; the search strategy used by the backtracking migration algorithm.
JSON	JavaScript Object Notation; the file format used for communication between Python and C++.
UI	User Interface; the layer through which the player interacts with the game.
HUD	Heads-Up Display; the on-screen area that would show game state information in a graphical interface.
NPC	Non-Player Character; a game entity controlled by the system instead of the player.
Pygame	Python library planned for graphical game rendering and interaction.
C++	Programming language used for the game engine and required data structures.
Python	Programming language used for the algorithms, JSON bridge, and console simulator.

Abstract

Ecosystem Survival is a grid-based strategy game prototype designed to simulate the management and preservation of species within a small ecosystem. The main objective of the game is to prevent species extinction by making decisions related to feeding and migration while dealing with limited actions, hunger levels, population changes, predator cells, food cells, and safe zones.

The project integrates Python and C++ components. The Python layer includes a greedy algorithm, a backtracking algorithm, a JSON bridge, a user interface layer, and a console simulator used for logic validation. The greedy algorithm recommends which species should be prioritized according to a risk score based on hunger and population. The backtracking algorithm searches for valid migration routes toward safe zones while avoiding predator cells, occupied cells, invalid positions, and repeated cells in the same route.

The C++ engine processes player actions and updates the official game state. It uses an AVL tree to store and update species data, including population, hunger, position, status, and feeding state. It also uses a linked list structure to represent migration path segments during movement processing. Communication between Python and C++ is handled through JSON files, mainly `state.json` and `input.json`.

The user interface presents the game state to the player and supports interaction with the ecosystem. It displays the board, species information, turn data, algorithmic recommendations, and action results. The console simulator complements this layer by allowing validation of the main gameplay logic, including feeding, migration, route inspection, turn progression, and win or loss conditions.

This report presents the design, methodology, implementation results, discussion, and conclusions of the project. It shows how algorithms, data structures, game rules, user interaction, and JSON communication were combined into an educational prototype for the Computer Sciences I course.

Keywords: greedy algorithm, backtracking, data structures, AVL tree, linked list, JSON bridge, game design, ecosystem simulation

GitHub repository

<https://github.com/JulianCarvajalG/ecosystem-survival>

Chapter 1

Introduction

In recent years, computational models and algorithmic approaches have become useful tools for representing decision-making problems in interactive systems. Games are especially appropriate for this purpose because they allow abstract concepts such as optimization, search, data structures, and state management to be connected with visible rules and player actions.

Within this context, *Ecosystem Survival* is proposed as a grid-based strategy game prototype that simulates the management of species in a simplified ecosystem. The player must prevent species extinction by deciding when to feed them and how to guide them toward safe zones. These decisions are affected by limited actions, hunger levels, population values, predator cells, food cells, safe zones, and movement constraints.

The project applies two main algorithmic techniques. A greedy algorithm is used to recommend which species should be prioritized according to an immediate risk score. A backtracking algorithm is used to search for valid migration routes through the grid while avoiding predator cells, occupied cells, invalid positions, and repeated cells in the same path.

The implementation also integrates two data structures required by the course project. A linked list is used in the C++ engine to represent migration path segments, while an AVL tree is used to store and update species information. Communication between the Python and C++ components is handled through JSON files, mainly `state.json` and `input.json`.

The current system includes implemented Python algorithms, a C++ engine, JSON state management, a user interface layer, and a console simulator used for logic validation. Together, these components demonstrate the main gameplay loop and show how algorithms and data structures can support an interactive ecosystem simulation.

1.1 Background

Ecosystem Survival is designed as a simulation-based game where species interact with a constrained environment represented as a grid. The game uses simplified ecological concepts, such as hunger, population, danger zones, food cells, and safe zones, to create a decision-making scenario.

From a computer science perspective, the project is relevant because it connects gameplay mechanics with algorithmic problems. Feeding decisions are modeled as a prioritization problem, while migration decisions are modeled as a constrained pathfinding problem. This makes the project useful for demonstrating how algorithms can support interactive behavior.

The project also shows how data structures can support state management. The AVL tree organizes species data in the C++ engine, while the linked list represents movement paths. The use of JSON files allows the Python and C++ parts of the system to communicate without being tightly coupled.

1.2 Problem Statement

Managing multiple species in a grid-based ecosystem requires decisions that depend on both priority and constraints. The player must decide which species needs immediate attention and which species can safely migrate toward a safe zone.

The first problem is prioritization. Each species has a hunger level and a population value. A species with high hunger and low population is more endangered than a species with low hunger and high population. Therefore, the system needs a way to calculate risk and recommend which species should be fed first.

The second problem is migration. A species must move through the board while avoiding predator cells and other occupied cells. The path must stay inside the grid and should not revisit the same position during one search. Therefore, the system needs a search strategy capable of finding a valid route to a safe zone.

The third problem is integration. The project uses Python for algorithms and interface support, while C++ is used for the engine and required data structures. These components need a simple and consistent way to exchange the game state and player actions.

1.3 Aims and Objectives

Aim: To develop a grid-based ecosystem survival prototype that applies algorithms, data structures, and JSON communication to support game decision-making.

Objectives:

- Implement a greedy algorithm to evaluate species risk and recommend feeding priority.
- Implement a backtracking algorithm to compute valid migration paths toward safe zones.
- Use a linked list to represent migration path information in the C++ engine.
- Use an AVL tree to store and update species data in the C++ engine.
- Use JSON files to exchange actions and game state between Python and C++.
- Provide a user interface layer that displays the game state and allows player interaction.
- Provide a console simulator to validate the main gameplay rules and algorithmic behavior.

1.4 Solution Approach

The solution follows a modular architecture. Python handles algorithmic support, route calculation, recommendations, JSON bridge functions, user interaction, and simulation support. C++ handles the official game state processing, including feeding, migration, turn effects, species updates, and win or loss validation.

The greedy algorithm evaluates each active species using the formula:

$$risk = hunger + (10 - population)$$

The species with the highest risk value is recommended as the most urgent one. Species marked as safe or extinct are ignored, and the feeding-specific recommendation also ignores species already fed during the current turn.

The backtracking algorithm searches for a route from a species position to a safe zone. It explores eight directions and validates each cell before adding it to the path. The complete path can be calculated, but migration is limited to a segment of at most two cells per action.

The C++ engine receives actions through `input.json` and writes the updated game state to `state.json`. This JSON-based approach separates the Python and C++ responsibilities while keeping the state easy to inspect and debug. The user interface uses this same state information to present the ecosystem to the player and send selected actions to the engine.

1.5 Scope

The scope of the current project includes the core gameplay logic, the Python algorithm layer, the JSON bridge, the C++ engine structures, the user interface layer, and the console simulator used for logic validation. The game uses a fixed 8 by 8 board and a small set of species.

The project includes feeding recommendations, migration pathfinding, turn management, hunger effects, population updates, safe-zone detection, extinction handling, win or loss conditions, user interaction, and JSON-based communication between components.

1.6 Assumptions

The following assumptions are made for the implemented system:

- The board size is fixed at 8 by 8.
- Species are represented by unique symbols.
- Predator cells are represented by X.
- Safe zones are represented by Z.
- Food cells are represented by C.
- The JSON files follow the expected structure.
- Python and C++ components use consistent relative paths for JSON communication.
- The player has two actions per turn.

1.7 Limitations

The project has several limitations:

- The ecosystem model is simplified and does not represent real ecological dynamics.
- The board size and initial species configuration are fixed in the current implementation.
- The greedy algorithm only considers the current state and does not guarantee long-term ecological balance.
- The backtracking algorithm may become expensive for larger boards with more obstacles.
- The graphical interface focuses on the main gameplay interactions and can be expanded with additional visual feedback, animations, and configuration options.
- The C++ engine depends on `json.hpp`, which must be available for successful compilation.

1.8 Summary of Contributions and Achievements

This project contributes a functional educational prototype that connects algorithms and data structures with game mechanics. The greedy algorithm supports feeding decisions, the backtracking algorithm supports migration, the AVL tree manages species data, and the linked list represents migration path information.

The implementation also demonstrates communication between Python and C++ through JSON files. The user interface provides the player-facing layer for interacting with the ecosystem, while the console simulator supports validation of the main game rules, including feeding, migration, route inspection, turn progression, hunger increase, population loss, extinction, and safe-zone victory conditions.

1.9 Organization of the Report

This report is organized as follows:

- Chapter 2 presents the theoretical background related to greedy algorithms, backtracking, data structures, JSON communication, and game systems.
- Chapter 3 describes the methodology, including system architecture, game rules, algorithms, data structures, JSON bridge, user interface, and validation strategy.
- Chapter 4 presents the implementation and results of the project.
- Chapter 5 discusses the strengths, limitations, scalability, and possible improvements of the project.
- Chapter 6 presents the conclusions and future work.

Chapter 2

Literature Review

2.1 Software Architecture for Digital Game Mechanics

Digital games and simulations are not only visual products; they are also software systems built around rules, state changes, and user interaction. In this sense, game mechanics require an architecture that separates responsibilities clearly. Mizutani, Daros, and Kon [1] explain that software architecture for digital game mechanics is important because mechanics are often specific to each game, but the organization of those mechanics can follow reusable architectural principles.

For *Ecosystem Survival*, this idea is relevant because the project combines several components: a C++ engine, Python algorithms, a user interface layer, and JSON files for communication. The game mechanics include feeding, migration, hunger increase, population changes, extinction, and safe-zone conditions. These mechanics must be processed consistently, while the user interface presents the current state and allows the player to make decisions.

A central architectural decision in the project is the separation between game logic and presentation. The C++ engine is responsible for processing the official state of the game, while Python supports algorithms, interaction, and visualization. This separation follows the general principle of decoupling discussed by Mizutani et al. [1]. By separating the logic from the interface, the project becomes easier to understand, test, and extend.

The use of JSON files also supports this architecture. The files `state.json` and `input.json` work as a simple communication contract between components. The state file represents the current condition of the ecosystem, and the input file represents the action selected by the player. This design allows the engine and the interface to exchange information without requiring direct access to each other's internal structures.

2.2 Algorithms in Game Decision-Making

Algorithms are commonly used in games to support decision-making, pathfinding, simulation, and state evaluation. In *Ecosystem Survival*, two algorithmic techniques are central to the gameplay logic: greedy selection and backtracking search.

The greedy algorithm is used to recommend which species should be fed first. According to the general definition of greedy algorithms, the system selects the locally best option according to a specific criterion [2]. In this project, the criterion is a risk score based on hunger and population:

$$risk = hunger + (10 - population)$$

This formula gives priority to species with high hunger and low population. The advantage of this approach is that it is simple, deterministic, and efficient. It can scan the list of species and produce a recommendation quickly. However, greedy algorithms have a known limitation: they do not necessarily produce a globally optimal solution because they focus on the current best option [2].

For this reason, the recommendation should be understood as immediate decision support, not as a complete ecological strategy.

2.3 Backtracking and Constrained Pathfinding

Backtracking is a search technique used when a solution must satisfy a set of constraints. It builds a possible solution step by step and abandons a path when it cannot lead to a valid result [2]. This is appropriate for the migration problem in *Ecosystem Survival* because a species must move through a grid while avoiding invalid positions.

In the project, the backtracking algorithm searches for a route from a species position to a safe zone. A valid cell must be inside the board, must not be a predator cell, must not be occupied by another species, and must not have been visited before in the same search. If the algorithm reaches a safe zone, it returns a valid route. If a path fails, it removes the last step and tries another direction.

This method fits the game because migration is not a simple direct movement. The board contains danger cells, occupied cells, and safe zones. Therefore, the algorithm must explore alternatives and respect constraints before selecting a route.

2.4 Data Structures for Game State Management

Data structures are necessary for organizing and updating the information used by a game system. In *Ecosystem Survival*, the main data structures are a linked list and an AVL tree.

A linked list is appropriate for representing a migration path because a path is naturally sequential. Each node can store one position of the route, and the next pointer preserves the order of movement. In the project, the C++ structure `MigrationPath` stores row and column values for migration segments.

An AVL tree is a self-balancing binary search tree. Its main advantage is that it keeps search, insertion, and update operations efficient by maintaining balance through rotations [2]. In the project, the AVL tree stores species information using the species symbol as the key. Each node contains data such as population, hunger, position, status, and feeding state.

Although the current number of species is small, the AVL tree is useful because it satisfies the course requirement and prepares the system for future scenarios with more species. It also shows how a formal data structure can be integrated into a game engine.

2.5 JSON as a Communication Contract

JSON is widely used as a lightweight format for representing structured data. In this project, JSON is used to connect Python and C++ components. This decision supports modularity because each component can focus on its own responsibility while sharing information through files.

The state file stores the board, turn number, remaining actions, species list, recommendation fields, migration path information, and game status. The input file stores the player action, the selected species, and, for migration, the path segment calculated by the backtracking algorithm.

This approach is simple and practical for an academic prototype. It also makes debugging easier because the current state can be inspected directly. However, it requires consistent field names, valid file paths, and correct JSON formatting.

2.6 Critique of the Review

The literature and theoretical concepts reviewed in this chapter support the main design decisions of *Ecosystem Survival*. The separation between engine, algorithms, interface, and data files follows

the architectural principle of decoupling described by Mizutani et al. [1]. The use of greedy and backtracking algorithms follows standard algorithmic techniques described by Cormen et al. [2].

However, these techniques also have limitations. The greedy algorithm is efficient but only evaluates the current state. It does not guarantee long-term ecological balance. Backtracking is flexible for constrained route search, but it can become expensive on larger boards. JSON communication is easy to inspect, but it depends on both languages using the same structure correctly.

These limitations do not invalidate the project. Instead, they define its academic scope. The project demonstrates how algorithms, data structures, and architecture can be combined in a clear educational prototype.

2.7 Summary

This chapter reviewed the main theoretical foundations of *Ecosystem Survival*: game architecture, greedy algorithms, backtracking, linked lists, AVL trees, and JSON communication. These concepts support the implementation of the project and explain why each component was selected.

The reviewed literature shows that modularity, separation of responsibilities, and structured data management are important for game systems. In this project, those principles are applied through a Python-C++ architecture connected by JSON files, with algorithms and data structures directly supporting the gameplay mechanics.

Chapter 3

Methodology

3.1 General Design Approach

The development of Ecosystem Survival follows an implementation-oriented methodology based on connecting game mechanics with algorithms, data structures, and file-based communication. The project was designed as a grid-based survival game where the player manages different species, feeds them when necessary, and guides them toward safe zones while avoiding predator cells.

The system is divided into three main parts. The first part is the Python layer, which contains the greedy algorithm, the backtracking algorithm, the JSON bridge, and the console simulator. The second part is the C++ engine, which processes actions, updates the game state, and uses the required data structures. The third part is the user interface layer, planned as a graphical Pygame interface that reads the game state and allows the player to interact with the system visually.

The methodology was based on decomposing the game into smaller computational problems. Feeding decisions were modeled as a priority problem solved with a greedy algorithm. Migration was modeled as a constrained pathfinding problem solved with backtracking. Species management was modeled using an AVL tree, and migration path representation was supported through a linked list.

The main implementation files considered in this methodology are:

- `game/algorithms/greedy.py`
- `game/algorithms/backtracking.py`
- `game/algorithms/bridge.py`
- `engine/main.cpp`
- `engine/linked_list.h`
- `engine/tree.h`
- `data/state.json`
- `simulator.py`

3.2 System Architecture

The architecture separates responsibilities between Python, C++, and JSON files. Python is responsible for algorithmic support and user-facing interaction. C++ is responsible for the official game state processing and the required data structures. JSON files are used as the communication bridge between both languages.

The C++ engine reads the current state from `state.json`, reads the player action from `input.json`, processes the action, updates the internal structures, and writes the new state back to `state.json`.

The Python layer reads this state, calculates recommendations or migration paths, and writes actions when the player decides what to do.

This architecture allows the project to keep the engine, algorithms, and interface separated. The interface does not directly modify the AVL tree or linked list. Instead, it communicates with the engine through JSON files. This makes the system easier to understand, debug, and extend.

3.3 Game State Representation

The game state is represented using the file `data/state.json`. This file stores all the information needed to describe the current situation of the game.

The main fields of the state are:

- `grid_size`: size of the board.
- `turn`: current turn number.
- `actions_left`: number of actions available in the current turn.
- `board`: two-dimensional representation of the ecosystem grid.
- `species`: list of species and their attributes.
- `greedy_recommendation`: field used to store or display a recommendation.
- `migration_path`: field used to represent migration path information.
- `game_status`: current state of the game.

The board is represented as an 8 by 8 grid. Each cell contains a symbol that identifies its content:

- `[]` represents an empty cell.
- `X` represents a predator or danger cell.
- `C` represents a food cell.
- `Z` represents a safe zone.
- `R`, `F`, and `T` represent species.

Each species stores its symbol, name, population, hunger level, position, and status. The implementation also uses the field `fed_this_turn` to control whether a species has already been fed during the current turn.

3.4 Game Rules and Turn System

The game follows a turn-based structure. Each turn gives the player two available actions. The player can feed a species, migrate a species, inspect a route, skip the turn, or quit the simulation.

Feeding reduces the hunger level of an active species. A species cannot be fed more than once in the same turn. This rule is controlled using the `fed_this_turn` field.

Migration uses a path calculated by the backtracking algorithm. The complete route may contain several cells, but a species can only move a maximum of two cells per migration action. This keeps movement gradual and forces the player to manage turns carefully.

At the end of each turn, active species become hungrier. If hunger reaches the defined threshold, population decreases. If the population of a species reaches zero, the species becomes extinct. The game is won when all non-extinct species reach safe zones. The game is lost if a species becomes extinct because its population reaches zero.

3.5 Greedy Algorithm Methodology

The greedy algorithm is implemented in `greedy.py`. Its purpose is to recommend which species should receive attention first. The algorithm evaluates the current condition of each species and selects the one with the highest immediate risk.

The implemented risk formula is:

$$risk = hunger + (10 - population)$$

This formula combines two important survival factors. Hunger represents urgency, while low population represents vulnerability. A species with high hunger and low population receives a higher risk score.

The algorithm scans the list of species, ignores species whose status is safe or extinct, computes the risk value for each valid species, and returns the species with the highest value.

The project also includes a feeding-specific version of the recommendation. This version ignores species that have already been fed during the current turn. This prevents the system from recommending an action that the player cannot legally perform.

The greedy approach is suitable for this mechanic because feeding is an immediate decision. The player needs fast guidance about the most endangered species in the current state, and the greedy algorithm provides a simple and deterministic recommendation.

3.6 Backtracking Algorithm Methodology

The backtracking algorithm is implemented in `backtracking.py`. Its purpose is to find a valid migration route from the current position of a species to a safe zone.

The algorithm explores eight possible movement directions: up, down, left, right, and the four diagonals. A movement is considered valid only if it satisfies the following conditions:

- the cell is inside the board,
- the cell has not been visited during the current search,
- the cell is not a predator cell marked as X,
- the cell is not occupied by another species.

The search starts at the selected species position. If the algorithm reaches a safe zone marked as Z, it returns the complete path. If a selected direction does not lead to a solution, the algorithm removes the last position from the path and tries another direction. This is the backtracking process.

The implementation also includes a function that limits migration to a maximum of two cells per action. This means that the algorithm can calculate the complete route, but the player only advances through a short segment during each migration action.

3.7 C++ Engine Methodology

The C++ engine is implemented in `engine/main.cpp`. Its responsibility is to process the official game state after each player action.

When the engine starts, it reads `data/state.json`. If an action exists in `data/input.json`, the engine reads the action type and the selected species. Then it loads the species data into an AVL tree, processes the action, updates the board, checks turn effects, and writes the new state back to `state.json`.

For feeding actions, the engine verifies that the selected species is active and has not already been fed during the current turn. If the action is valid, hunger is reduced and the species is marked as fed.

For migration actions, the engine receives a migration path segment generated by the Python backtracking algorithm. The species is moved to the last cell of that segment. If the destination is a safe zone, the species status becomes safe. If the species passes through a food cell, hunger is reduced.

After each action, the engine decreases the number of actions left. When the turn ends, it increases hunger for active species, decreases population when hunger reaches the threshold, checks extinction, resets feeding markers, verifies win or loss conditions, and writes the updated state.

3.8 Linked List Methodology

The linked list is implemented in `engine/linked_list.h`. The structure is called `MigrationPath`. Each node stores a row and a column, representing one position in a migration path.

The linked list supports appending positions, clearing the path, checking its size, checking whether it is empty, and converting the path into a vector.

This structure is appropriate for migration because a path is naturally sequential. Each node represents one step, and the order of the nodes represents the order of movement. In the engine, the linked list is built from the migration segment received from Python.

In the current implementation, the linked list is used during migration processing. It demonstrates the required data structure and represents the path segment traveled by the species.

3.9 AVL Tree Methodology

The AVL tree is implemented in `engine/tree.h`. It stores species information using the species symbol as the key. Each node stores the species symbol, name, population, hunger, position, status, feeding state, height, and child pointers.

The AVL tree includes insertion, search, update, balancing, rotations, and inorder traversal. The engine loads the species from the JSON state into the AVL tree, updates them during action processing, and serializes the updated species data back into JSON.

The AVL tree is useful because the engine needs to search and update species by symbol. Although the current number of species is small, the AVL tree satisfies the course requirement and prepares the system for future versions with more species.

3.10 JSON Bridge Methodology

The JSON bridge is implemented in `game/algorithms/bridge.py`. Its purpose is to connect the Python layer with the C++ engine through file input and output.

The function `read_state` reads the current game state from `state.json`. The function `write_action` writes the player action into `input.json`. For feeding actions, the input file contains the action type and the selected species. For migration actions, it also includes the migration path segment calculated by backtracking.

This file-based communication model keeps the system simple. Python can calculate recommendations and routes, while C++ can process the official game state. Both sides only need to agree on the JSON structure.

3.11 User Interface Methodology

The user interface is designed as the visual and interactive layer of Ecosystem Survival. Its purpose is to help the player understand the current ecosystem state and execute actions in an intuitive way.

The interface is implemented in Python and works together with the algorithmic layer and the JSON bridge. It reads the current game state, displays the board, shows species information, presents recommendations, and sends player actions to the engine.

The central element of the interface is the 8 by 8 board. Each cell represents one position in the ecosystem. The UI displays empty cells, predator cells, food cells, safe zones, and species using clear visual symbols or colors.

The interface also includes an information panel with the current turn, available actions, species names, population values, hunger levels, positions, and statuses. This information is necessary because the player must decide which species to feed or migrate based on the current risk.

The greedy recommendation is displayed as a suggested species to feed. This helps the player identify the species with the highest immediate risk. The backtracking result is shown as a migration route or as the next movement segment toward a safe zone.

The interaction model is based on selecting an action and a species. When the player chooses to feed a species, the UI writes the corresponding action to `input.json`. When the player chooses to migrate a species, the UI uses the backtracking result and writes the movement segment to the JSON input file.

The UI does not directly modify the C++ data structures. Instead, it communicates through the JSON bridge. This keeps the visual layer separated from the engine and makes the system easier to maintain.

3.12 Console Simulator Methodology

The repository also includes `simulator.py`, which implements a console-based version of the gameplay loop. This simulator imports the greedy and backtracking algorithms and allows the team to validate the main mechanics through text-based interaction.

The simulator allows the player to:

- view the board,
- view species data,
- receive greedy recommendations,
- inspect migration routes,
- feed species,
- migrate species,
- skip turns,
- trigger win or loss conditions.

The console simulator is useful for validating the logic of the project alongside the graphical interface. It confirms that the main algorithms can be used directly during gameplay and helps test the rules before integrating or modifying visual behavior.

3.13 Validation Methodology

The validation process was based on checking the consistency between the source code, the JSON state, and the expected game rules.

The greedy algorithm was validated by confirming that it calculates the risk score and selects the species with the highest value. The backtracking algorithm was validated by confirming that it avoids predator cells, occupied cells, invalid positions, and already visited cells.

The JSON bridge was validated by checking that it reads `state.json` and writes `input.json` using the expected fields. The C++ engine was validated by inspecting how it processes feeding and migration actions, updates species data, applies turn effects, and writes the updated state.

The data structures were validated by inspecting their implementation and their role in the engine. The linked list represents migration path segments, while the AVL tree stores and updates species data.

3.14 Summary

The methodology of Ecosystem Survival is based on separating the project into clear components. The greedy algorithm supports feeding decisions, the backtracking algorithm supports migration, the linked list represents movement paths, the AVL tree manages species data, and JSON files connect Python with C++.

The console simulator provides a playable way to test the core logic, while the planned Pygame interface provides the visual layer for the final game experience. Together, these components form a coherent educational prototype that applies algorithms and data structures to an interactive ecosystem survival scenario.

Chapter 4

Implementation and Results

4.1 Implemented Components

The final repository contains the main algorithmic and structural components of the Ecosystem Survival project. The Python layer includes the greedy algorithm, the backtracking algorithm, and the JSON bridge. The C++ layer includes the game engine, a linked list for migration paths, and an AVL tree for species data.

The implemented Python files are located in `game/algorithms`. The file `greedy.py` recommends which species should receive attention first. The file `backtracking.py` searches for valid migration routes toward safe zones. The file `bridge.py` defines the communication functions used to read the game state and write player actions through JSON files.

The C++ engine is located in the `engine` folder. The file `main.cpp` reads the current game state, processes actions, updates the internal structures, applies turn effects, and writes the updated state. The file `linked_list.h` defines a linked list structure for migration paths. The file `tree.h` defines an AVL tree for storing and updating species information.

The repository also includes `simulator.py`, a console-based playable simulator. This simulator confirms the main gameplay logic without requiring a graphical interface.

4.2 Gameplay From the User Point of View

From the confirmed implementation, the player interacts with the game through the console simulator. The simulator displays the board, the current turn, the remaining actions, the species information, the greedy recommendation, and the available migration routes.

The player can use commands such as `feed R`, `migrate F`, `ruta T`, `skip`, and `quit`. Feeding reduces the hunger value of an active species, as long as that species has not already been fed in the current turn. Migration calculates a route toward a safe zone and moves the species only through a limited segment of the route.

Each turn has two actions. When the actions are used, the turn ends. At the end of the turn, hunger increases for active species. If hunger reaches the defined threshold, population decreases. If population reaches zero, the species becomes extinct. The game is won when all non-extinct species reach safe zones.

4.3 Greedy Algorithm Result

The greedy algorithm is used to recommend the species that should be fed first. The implementation calculates a risk value using the following expression:

$$risk = hunger + (10 - population)$$

This means that a species with high hunger and low population receives a higher risk score. The algorithm ignores species whose status is safe or extinct. The feeding-specific recommendation also ignores species that have already been fed during the current turn.

In the simulator, this result is shown as a recommendation to the player. The recommendation does not force the player's decision, but it gives useful guidance about which species is in the most critical condition according to the current state.

4.4 Backtracking Algorithm Result

The backtracking algorithm is used to find migration routes from a species position to a safe zone. The implementation explores eight possible movement directions: vertical, horizontal, and diagonal. A cell is considered valid only if it is inside the board, has not already been visited in the current search, is not a predator cell, and is not occupied by another species.

Predator cells are represented with X, while safe zones are represented with Z. If the algorithm reaches a safe zone, it returns the route. If no valid route exists, it returns an empty list.

The simulator can show the complete route with the `ruta` command. During an actual migration action, the species does not move through the complete route immediately. Instead, the function `get_next_steps` limits the movement to a maximum of two cells per action. This makes migration gradual and keeps the turn system relevant.

4.5 Contribution of the C++ Engine

The C++ engine contributes the state-processing layer of the project. It reads `data/state.json`, loads species data into an AVL tree, reads the player action from `data/input.json`, processes the action, applies end-of-turn effects when needed, checks the game status, and writes the updated state back to `state.json`.

For feeding actions, the engine reduces hunger and marks the species as already fed during that turn. For migration actions, the engine receives a migration segment calculated in Python and moves the species to the last position of that segment. If the species reaches a safe zone, its status becomes safe.

The engine also creates a `MigrationPath` linked list from the received migration segment. This satisfies the data structure requirement by representing the traveled path as a sequence of connected nodes. However, in the inspected implementation, this linked list is created inside the migration action scope and is not persisted after the action finishes.

4.6 Use of JSON Files

The project uses JSON files as a bridge between Python and C++. The file `state.json` stores the current game state, including the board, turn number, remaining actions, species data, migration path, greedy recommendation field, and game status.

The file `input.json` is used to send actions from Python to the C++ engine. For a feeding action, it stores the action type and the species symbol. For a migration action, it also stores the migration path segment calculated by the backtracking algorithm.

This design makes the communication simple and easy to inspect. It also allows the Python and C++ components to remain separated. However, it depends on consistent file paths, correct field names, and valid JSON structure.

4.7 Validation

The validation was based on inspection of the implemented source code and the available repository files. Table 4.1 summarizes the main validation points.

Validation item	Status	Evidence
Greedy selects highest-risk species	Passed	<code>greedy.py</code> risk formula
Greedy ignores safe/extinct species	Passed	Status checks in <code>greedy.py</code>
Greedy avoids repeated feeding	Passed	<code>fed_this_turn</code> validation
Backtracking avoids predator cells	Passed	X rejected in <code>is_valid_cell</code>
Backtracking avoids cycles	Passed	Visited matrix is used
Backtracking avoids occupied cells	Passed	Other species symbols are rejected
JSON bridge reads state file	Passed	<code>read_state</code> in <code>bridge.py</code>
JSON bridge writes action file	Passed	<code>write_action</code> in <code>bridge.py</code>
Linked list stores migration segment	Partially passed	Used in <code>main.cpp</code>
AVL tree stores species data	Passed	Implemented in <code>tree.h</code>
User interface displays game state	Passed	Python UI layer presents board and species data
Console simulator validates game logic	Passed	Implemented in <code>simulator.py</code>
C++ engine integration	Partially passed	Requires <code>json.hpp</code> dependency

Table 4.1: Validation summary based on the inspected repository.

The validation shows that the core algorithms, data structures, user interaction layer, JSON bridge, and simulator are represented in the project. The greedy algorithm, backtracking search, AVL tree, linked list, and JSON communication all support specific gameplay responsibilities.

The user interface provides the player-facing layer for displaying the ecosystem state and sending actions through the JSON bridge. The console simulator supports validation by reproducing the main gameplay loop in a text-based environment.

The main integration limitation is the dependency management of the C++ engine. The file `main.cpp` includes `json.hpp`, so this header must be available for successful compilation.

Chapter 5

Discussion

5.1 Interpretation of the Implementation

The implementation of Ecosystem Survival shows how algorithms and data structures can be connected to concrete gameplay mechanics. The project does not use the greedy algorithm, backtracking, the linked list, or the AVL tree as isolated exercises. Instead, each element supports a specific part of the game logic.

The greedy algorithm supports feeding decisions. The backtracking algorithm supports migration decisions. The AVL tree stores and updates species data inside the C++ engine. The linked list represents migration path information during movement processing. Finally, the JSON bridge allows Python and C++ to exchange information through a simple file-based communication model.

This integration is one of the main strengths of the project. The game rules are simple, but they are enough to demonstrate how computer science concepts can be applied in an interactive system.

5.2 Greedy Algorithm Analysis

The greedy algorithm is appropriate for immediate feeding decisions because the problem requires selecting one species that needs attention at the current moment. The implemented risk formula considers both hunger and population:

$$risk = hunger + (10 - population)$$

This formula gives higher priority to species that are hungry and have a low population. In this context, a greedy strategy is useful because the player has limited actions per turn and needs quick guidance.

However, the greedy algorithm has an important limitation. It only evaluates the current state of the ecosystem. It does not predict future hunger values, future migration opportunities, or the long-term balance between species. Therefore, the greedy recommendation can identify the most urgent short-term case, but it does not guarantee the best global strategy for the entire game.

This limitation is acceptable for the project scope because the goal is to demonstrate a clear greedy decision rule, not to solve the complete ecosystem planning problem optimally.

5.3 Backtracking Analysis

Backtracking is appropriate for the migration problem because movement is constrained by the board. A species cannot move outside the grid, cannot pass through predator cells, cannot move through cells occupied by other species, and should not revisit the same cell during one search.

The implemented algorithm explores possible paths recursively. When a path cannot continue, the algorithm returns to a previous position and tries another direction. This behavior matches the

migration requirement because the system must search for a valid route instead of simply moving in a straight line.

The algorithm also recalculates the route using the current board state. This is important because the board can change after each action. A route that was valid in a previous state may no longer be valid after another species moves.

A limitation of this approach is scalability. Backtracking can become expensive on larger boards because it may explore many possible paths before finding a safe zone or concluding that no route exists. For the current 8 by 8 board, this is acceptable. For larger scenarios, a breadth-first search or a heuristic search could be more efficient.

5.4 Linked List Analysis

The linked list is suitable for representing a migration path because a path is naturally sequential. Each node can store one board position, and the next pointer represents the order of movement.

In the inspected implementation, the C++ class `MigrationPath` stores row and column values in linked nodes. It supports appending new positions, clearing the path, checking its size, and converting it to a vector. This satisfies the basic requirement of using a linked list to represent migration path data.

However, the linked list is used only temporarily inside the migration action processing in `main.cpp`. After the action finishes, the local linked list is destroyed. This means the structure demonstrates the required concept, but it is not used as a persistent part of the game state. A future version could store the last migration path in the engine state or serialize it into `state.json` for visualization.

5.5 AVL Tree Analysis

The AVL tree is appropriate for species data because the engine needs to search and update species by symbol. Each species contains several fields, including population, hunger, position, status, and feeding state.

The implementation in `tree.h` includes insertion, search, update, rotations, height management, and inorder traversal. These operations show the main behavior expected from an AVL tree. The tree keeps the species organized by symbol and allows the engine to retrieve a species when processing an action.

In the current game, the number of species is small, so the performance advantage of an AVL tree is not essential. A simple list would also work for three species. However, the AVL tree is still justified because it satisfies the course requirement and prepares the design for future versions with more species.

5.6 JSON Bridge Analysis

The JSON bridge is a practical solution for communication between Python and C++. The Python layer can read the current state from `state.json`, calculate recommendations or paths, and write actions to `input.json`. The C++ engine can then read those actions, update the game state, and write the result back to `state.json`.

This design has several advantages. It is easy to inspect, easy to debug, and does not require complex networking or inter-process communication. It also separates responsibilities between the algorithm layer and the engine layer.

At the same time, the JSON bridge has limitations. Both sides must use the same file paths and field names. If a file is missing, malformed, or outdated, the game can fail or behave incorrectly. For a larger project, the bridge could be improved with validation functions, clearer error handling, and automatic generation of an initial `input.json` when needed.

5.7 Technical Challenges

The main technical challenge in the project is integration between the graphical interface, the Python algorithmic layer, the JSON bridge, and the C++ engine. Each component has a clear responsibility, but the complete system depends on consistent communication between them.

The user interface must read the current state from `state.json`, display the board and species information, and write player actions to `input.json`. The engine must then process those actions and write an updated state. Because of this, the field names, file paths, and update order must remain consistent across the system.

Another challenge is keeping the algorithmic results synchronized with the current board state. The greedy recommendation and migration route should be calculated using the latest available state. If the state is outdated, the recommendation or route may no longer match the real situation of the game.

The C++ engine also depends on the external `json.hpp` header from the nlohmann JSON library. This dependency must be included or installed correctly so that the engine can compile and process JSON files.

These challenges are common in multi-language projects. The architecture is understandable and modular, but it requires careful coordination between Python, C++, and JSON data.

5.8 Scalability Considerations

For the current board size, the algorithms are reasonable. The greedy algorithm is efficient because it only scans the species list. The AVL tree is also efficient for search and update operations. The backtracking algorithm works for the small grid, but it is the component most likely to face scalability problems.

If the board becomes larger or contains more obstacles, backtracking may explore many paths. This could slow down migration decisions. A future version could use breadth-first search to find shorter paths or A* search to guide movement toward safe zones more efficiently.

If the number of species increases, the AVL tree becomes more useful. It would allow faster organization and access than an unsorted list. The JSON files would also need stricter validation because larger states are more likely to contain inconsistent data.

5.9 Future Improvements

Several improvements could strengthen the project. The graphical interface could be expanded with additional visual feedback, animations, clearer route highlighting, and more detailed status indicators for each species.

A second improvement would be adding automated tests for the greedy and backtracking algorithms. These tests could confirm that the greedy algorithm selects the expected species and that backtracking avoids predator cells in different board configurations.

A third improvement would be improving the C++ integration by including the required `json.hpp` dependency and documenting the exact compilation command. This would make the engine easier to reproduce.

Finally, the linked list could be used more visibly by storing the latest migration path in the game state. This would make the data structure contribution clearer during execution and could support route visualization in the interface.

Chapter 6

Conclusions

The development of Ecosystem Survival demonstrates how fundamental computer science concepts can be integrated into a functional game prototype. The project combines algorithms, data structures, game rules, user interaction, and file-based communication to create an interactive ecosystem simulation where the player must protect different species from extinction and guide them toward safe zones.

The project achieved its main objective by connecting theoretical course topics with practical implementation. The greedy algorithm supports immediate feeding decisions, the backtracking algorithm searches for valid migration routes, the AVL tree stores species data in the C++ engine, the linked list represents migration path information, and JSON files allow communication between the Python and C++ components.

Key conclusions from the project are as follows:

- **Integration of Algorithms and Gameplay:** The project successfully connects algorithmic logic with player decisions. The greedy algorithm recommends the species with the highest immediate risk, while the backtracking algorithm identifies possible migration routes toward safe zones.
- **Use of Data Structures:** The implementation includes a linked list for migration path representation and an AVL tree for species information. These structures support the technical requirements of the project and show how abstract data structures can be applied to game state management.
- **JSON-Based Communication:** The use of `state.json` and `input.json` provides a simple bridge between Python and C++. This approach makes the game state easy to inspect and allows each language to focus on specific responsibilities.
- **User Interaction:** The project includes a user interface layer that presents the game state and allows the player to interact with the ecosystem. The console simulator also supports validation of the core logic and gameplay rules.
- **Course Requirement Fulfillment:** The implementation satisfies the main technical requirements of the project: greedy decision making, backtracking search, linked list usage, AVL tree usage, JSON communication, and an interactive simulation of the ecosystem.
- **Implementation Limitations:** Some aspects can still be improved, especially dependency management, automated testing, visual feedback, and scalability for larger boards or more species.
- **Educational Value:** The project helped reinforce the relationship between algorithms, data structures, software architecture, and user interaction. It also showed the importance of integration, documentation, validation, and clear communication between different parts of a system.

In summary, Ecosystem Survival fulfills its role as both a technical and educational project. It provides a clear example of how greedy algorithms, backtracking, linked lists, AVL trees, JSON communication, and user interaction can be applied in an interactive game context. Future work should focus on strengthening automated tests, improving dependency management, expanding interface feedback, and adding more species, board configurations, and ecological events.

References

- [1] W. K. Mizutani, V. K. Daros, and F. Kon, "Software Architecture for Digital Game Mechanics: A Systematic Literature Review," *Entertainment Computing*, vol. 38, p. 100421, 2021. Available: <https://doi.org/10.1016/j.entcom.2021.100421>
- [2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA, USA: MIT Press, 2009.